Aeronautics and Space Technology Club

IIT (ISM) Dhanbad

Signal Processing Ladder: A Hands-On Problem Series from Sensors to Machine Learning

Problem Statement 16 · Part D: Digital Signal Processing Level 3

Track: Decimation, Interpolation, Polyphase Filters, CIC Filters, Digital Down-Conversion, IQ Signals

Prerequisites: Problems 12, 14 · **Estimated effort:** 12–16 hours

Multirate Processing: Throwing Samples Away Without Throwing Information Away

A software-defined radio delivers millions of samples per second of which you need thousands. Implement decimation and interpolation with anti-alias and anti-image filters, polyphase structures that do the work at the low rate, rational resampling, an integer CIC filter, and a complete digital down-converter that pulls a narrowband FM signal out of a wideband IQ capture.

General Instructions

- **Thorough Review:** Read the whole problem statement, including the theory, before writing any code. Do the short analytic derivations *by hand first*; the code is there to confirm and extend your understanding, not to replace it.
- **Pacing:** This is a self-paced problem. The split below is a guide for the effort estimate in the header.

Stage	Phase	What you should have at the end of it
1	Think (∼20%)	Theory understood; every analytic result derived; signals, units and sampling rates written down.
2	Build (∼30%)	Core code working end to end on the baseline data and validated against the analytic results.
3	Experiment (∼30%)	Full sweeps, stress tests and the real or synthetic datasets processed; first plots and numbers.
4	Refine (∼20%)	Weaknesses fixed, honest analysis and memo written, repository folder clean.

- **Compute and Tools:** Everything is designed to run on a laptop CPU in Python 3.10+ (NumPy, SciPy, Matplotlib, and where stated pandas, Astropy, scikit-learn, PyWavelets and PyTorch). No GPU and no hardware are required; where a problem offers an optional hardware experiment it is marked as a bonus. A full run should never need more than a few minutes; if yours does, vectorise before you scale up.
- **Data:** Each problem lists (i) a *synthetic generator* from `sp_common.synth` (offline, seeded, with known ground truth) and (ii) *real datasets* to download yourself. Develop and validate on the synthetic data first, then repeat the key result on real data. Never commit large raw data files to the repository.
- **Submission Format:** Push your work to `solutions/16_multirate_processing/` in the repository: (i) clean, commented, reproducible Jupyter notebook(s) with fixed random seeds, (ii) a

short written report (`report.pdf` or `report.md`) containing the requested analysis sections, and (iii) all figures in a `figures/` sub-folder. The report must be understandable without running the code.

- **Originality:** All code, analysis and written explanations must be your own work. Any external library, snippet, paper or AI tool you use must be cited in the report. You should be able to explain every line you submit.
- **Honesty over Hype:** Validate against analytic results *before* trusting a number. A modest or negative result that is correctly analysed is worth more than an inflated claim. Report error bars, false-alarm rates and failure cases, not only your best run.
- **Units:** Write the units of every quantity and state in the first notebook cell whether a frequency is in Hz or rad/s and whether it is normalised to the sampling rate. Most mistakes in signal processing are factors of 2π, factors of 2 in one-sided spectra, or an off-by-one in the frequency axis.
- **Getting Stuck:** Re-read the theory section, check the self-check targets, then ask your mentor or open an issue in the repository. Describe what you expected, what you got, and what you already tried.

Background and Motivation

A low-cost SDR dongle samples a 2.4 MHz-wide slice of spectrum, 2.4 million complex samples per second, 76.8 Mbit/s at 16 bits per component. The signal you want, a 12.5 kHz narrowband FM channel or a 9.6 kbaud telemetry stream, is a thousandth of that. You must select it, filter, and reduce the rate so that the rest of the chain (demodulation, decoding) runs on a small processor. Doing this naively, a long filter running at the full rate, is wasteful by two to three orders of magnitude; multirate structures do the work at the low rate.

The same ideas run in reverse for interpolation (a DAC fed from a slow data stream), for sample-rate conversion between asynchronous clocks, and inside sigma-delta converters (problem 10). This problem builds each block and then assembles the digital down-converter used by every SDR receiver.

Theoretical Framework

1. Decimation and Interpolation

Keeping every Mth sample replicates the spectrum with spacing f_s/M, so aliasing must be prevented by low-pass filtering with cutoff $f_s/(2M)$ *before* downsampling. Interpolation by L inserts $L-1$ zeros between samples (creating $L-1$ spectral images) and removes the images with a low-pass filter of cutoff $f_s/2$ (original rate) and gain L. Rational resampling by L/M is interpolation followed by decimation with a single filter of cutoff $\min(\pi/L, \pi/M)$ at the intermediate rate.

2. Polyphase Structures

Writing the filter as M sub-filters $h_p[n] = h[nM + p]$ lets a decimator compute only the output samples that are kept: a filter of K taps then costs K multiplications per *output* sample instead of per *input* sample, a saving of a factor M. Likewise an interpolator needs K/L multiplications per output sample. The noble identities allow moving the rate change through the filter.

3. CIC Filters

A cascaded integrator–comb filter of order N, rate change R and differential delay 1 has

$$H(z) = \left[\frac{1 - z^{-R}}{1 - z^{-1}}\right]^N, \qquad |H(\mathrm{e}^{\mathrm{j}\omega})| = \left|\frac{\sin(\omega R/2)}{\sin(\omega/2)}\right|^N, \qquad \text{DC gain } R^N, \tag{1}$$

requiring no multipliers. The integrators overflow, but two's-complement wrap-around cancels in the combs provided the register width is at least $N \log_2 R + B_{\text{in}}$ bits (bit growth). Its passband droops as sinc^N, usually corrected by a short compensation FIR after decimation.

4. Complex Baseband and Digital Down-Conversion

A real band-pass signal at f_c is represented by the complex baseband $x_{\text{bb}}[n] = x[n]\mathrm{e}^{-\mathrm{j}2\pi f_c n/f_s}$ after low-pass filtering; negative frequencies are meaningful for complex (IQ) data. A *digital down-converter* (DDC) mixes the wanted channel to DC, low-pass filters, and decimates, typically in stages (CIC first, then FIR). An FM signal in complex baseband is demodulated by $\Delta\phi[n] = \arg\big(x[n]\,x^*[n-1]\big)$, proportional to the instantaneous frequency.

Data and Simulation Specification

Wideband IQ	`sp_common.synth.iq_multiband(fs=2.4e6, duration=0.25)`: NBFM voice-like signal at +200 kHz (audio tones 1 kHz and 2.3 kHz), 9.6 kbaud BPSK-like data at −350 kHz, a CW tone at +50 kHz and a DC spike.
Decimation	Test tones near $f_s/(2M)$ for $M = 2, 4, 8$; Kaiser anti-alias FIR with 60 dB stopband.
Rational resampling	44.1 → 48 kHz ($L = 160$, $M = 147$) of a swept sine.
CIC	$R = 8, 16$, $N = 3, 4$, 16-bit signed input; integer arithmetic with wrap-around.
DDC chain	2.4 MS/s → 50 kS/s (total decimation 48), CIC decimation by 12 then FIR by 4.
Real data (optional)	Your own capture from an RTL-SDR dongle of a broadcast FM station (centre frequency 88–108 MHz, 2.4 MS/s, 8-bit IQ); demodulate the audio.

Your Task

1. **Decimation (Stage 2).** Decimate a tone close to the new Nyquist frequency by $M = 2, 4, 8$ with and without an anti-alias filter and show the alias. Design the filter with Kaiser's formula, implement filter-then-downsample and a polyphase decimator, verify that they give identical outputs, and count multiplications per output sample to confirm the saving of M.
2. **Interpolation.** Interpolate a sine by $L = 4$ and $L = 8$ by zero-stuffing and filtering. Plot the images before filtering and verify the required gain L. Compare the interpolation error with linear interpolation and with sinc interpolation of problem 02.
3. **Rational resampling.** Resample a swept sine from 44.1 to 48 kHz with a hand-built polyphase resampler; compare with `scipy.signal.resample_poly` and measure the SNR of the difference. Explain how the filter length is chosen at the intermediate rate of 7.056 MHz.
4. **CIC filter.** Implement integrator and comb sections in integer arithmetic. Verify the frequency response of Eq. (1), the DC gain R^N and the first nulls at multiples of f_s/R. Demonstrate wrap-around correctness with a register width equal to the bit-growth formula and the failure with fewer bits. Design a droop-compensation FIR.
5. **Digital down-converter.** From the IQ capture extract the NBFM channel at +200 kHz: mix to DC, decimate with a CIC and an FIR, FM-demodulate and recover the 1 kHz and 2.3 kHz

audio tones. Plot the spectrum at every stage and report the SNR of each recovered tone. Compare the total multiplication count with a single-stage FIR running at 2.4 MS/s.

6. **Channeliser.** Extract all three signals. For the BPSK data, resample to an integer number of samples per symbol (for example 4 or 8) with your polyphase resampler, apply a matched filter and recover the bit stream; compare with the transmitted bits (the generator's seed gives the truth if you regenerate it).
7. **Real capture and CPU budget.** Process your own RTL-SDR capture (or, failing that, tone-generator data through your sound card) with the same chain. Tabulate the multiply–accumulate operations per second of every stage and state whether the chain would run on a Raspberry Pi-class processor in real time.

Self-check targets

A CIC filter with $R = 8$, $N = 3$ has DC gain $8^3 = 512$ and nulls at $f_s/8$ and its multiples; the bit growth is $N \log_2 R = 9$ bits. A polyphase decimator by M needs $1/M$ of the multiplications per input sample of the direct filter. The rational factor for 44.1 → 48 kHz is $L/M = 160/147$. The DDC must recover the 1 kHz and 2.3 kHz tones at the right frequencies after FM demodulation (check the spectrum peaks to < 1 Hz).

Aerospace Application Link

A ground station records 2.4 MS/s IQ (76.8 Mbit/s at 16 bit per component) while the satellite telemetry occupies 9.6 kbaud. Using your CPU-budget table compute the data rate and MAC rate before and after the DDC, the storage per pass of 10 min in each case and the minimum processor class that can run the chain in real time. Explain why a CIC stage is attractive on an FPGA and where a polyphase FIR is better on a CPU.

Deliverables

D1 **Decimation:** aliasing demonstration and polyphase-versus-direct equivalence with operation counts.

D2 **Interpolation and rational resampling:** images, gain check and the 160/147 resampler with SNR versus SciPy.

D3 **CIC filter:** integer implementation, response check, bit-growth experiment and compensation filter.

D4 **DDC:** the three-stage chain with spectra at each stage and recovered audio tones.

D5 **Channeliser and real data:** BPSK bit recovery and the CPU-budget table.

D6 **Memo (250 words):** "How much processing does a one-signal ground-station receiver need?" Cite at least two external sources.

Bonus (a) Build a polyphase FFT channeliser that extracts all channels at once and compare its cost with separate DDCs. (b) Implement a Farrow fractional-delay resampler for asynchronous clock correction. (c) Capture ADS-B at 1090 MHz and decode a frame.

Learning Resources

Multirate Signal Processing

- Crochiere, R.E., Rabiner, L.R. *Multirate Digital Signal Processing.*
- Vaidyanathan, P.P. *Multirate Systems and Filter Banks.*
- Oppenheim, A.V., Schafer, R.W. *Discrete-Time Signal Processing*, Ch. 4.6–4.7 (changing the sampling rate) and Ch. 12 (polyphase).
- Hogenauer, E.B. (1981). "An economical class of digital filters for decimation and interpolation." IEEE Trans. Acoust. Speech Signal Process. 29, 155.

Software-Defined Radio

- Lyons, R.G. *Understanding Digital Signal Processing*, Ch. 10 (sample-rate conversion) and Ch. 8 (quadrature signals).
- RTL-SDR guides: `https://www.rtl-sdr.com`; GNU Radio tutorials: `https://wiki.gnuradio.org`

Tools

- SciPy `signal.resample_poly`, `signal.decimate`, `signal.firwin`: `https://docs.scipy.org/doc/scipy/reference/signal.html`

Evaluation

Score your own submission out of 100 on five equally weighted criteria, then ask your mentor to score it independently and compare.

Criterion	What to look for	Marks
Conceptual depth	Correct mathematics and physics; every result explained through a mechanism (why the spectrum looks like this), not only through code behaviour.	/20
Implementation quality	Correct, modular, commented, reproducible code; no unexplained magic numbers; analytic checks and self-check targets pass.	/20
Aerospace-application integration	The signal-processing result is carried through to a measurable consequence for a sensor, a link, a flight system or a detection decision, as the task requires.	/20
Analysis and critical thinking	Error bars, convergence, comparison to theory, honest treatment of limitations, false alarms and failure cases.	/20
Communication	Clear report, labelled plots (units, legends, captions) and a memo that a non-expert student could follow.	/20

Bonus tasks are not part of the 100 marks. A bonus attempted badly does not cost you marks, and a well-executed one is the best sign that you own the material.